or “steal” its contents without affecting program correctness. This means that rather than *copying* the contents of an rvalue parameter, you can just *move* the contents. For large dynamic structures, this saves a lot of memory allocation and provides a lot of scope for optimization. Consider a function that takes a `std::vector<int>` as a parameter and needs to have an internal copy for modification, without touching the original. The old way of doing this would be to take the parameter as a `const` lvalue reference and make the copy internally:

```
void process_copy(std::vector<int> const& vec_)
{
     std::vector<int> vec(vec_);
     vec.push_back(42);
}
```

This allows the function to take both lvalues and rvalues but forces the copy in every case. If you overload the function with a version that takes an rvalue reference, you can avoid the copy in the rvalue case, because you know you can freely modify the original:

```
void process_copy(std::vector<int> && vec)
{
     vec.push_back(42);
}
```

Now, if the function in question is the constructor of your class, you can pilfer the innards of the rvalue and use them for your new instance. Consider the class in the following listing. In the default constructor it allocates a large chunk of memory, which is freed in the destructor.

Listing A.1. A class with a move constructor